DatumLex: automação de deploy do GitHub para a Render

Data: 24 de outubro de 2026

Escopo: publicação automática do código da aplicação.

Objetivo

O DatumLex conecta o repositório GitHub à Render para publicar alterações aprovadas sem enviar manualmente os arquivos da aplicação.

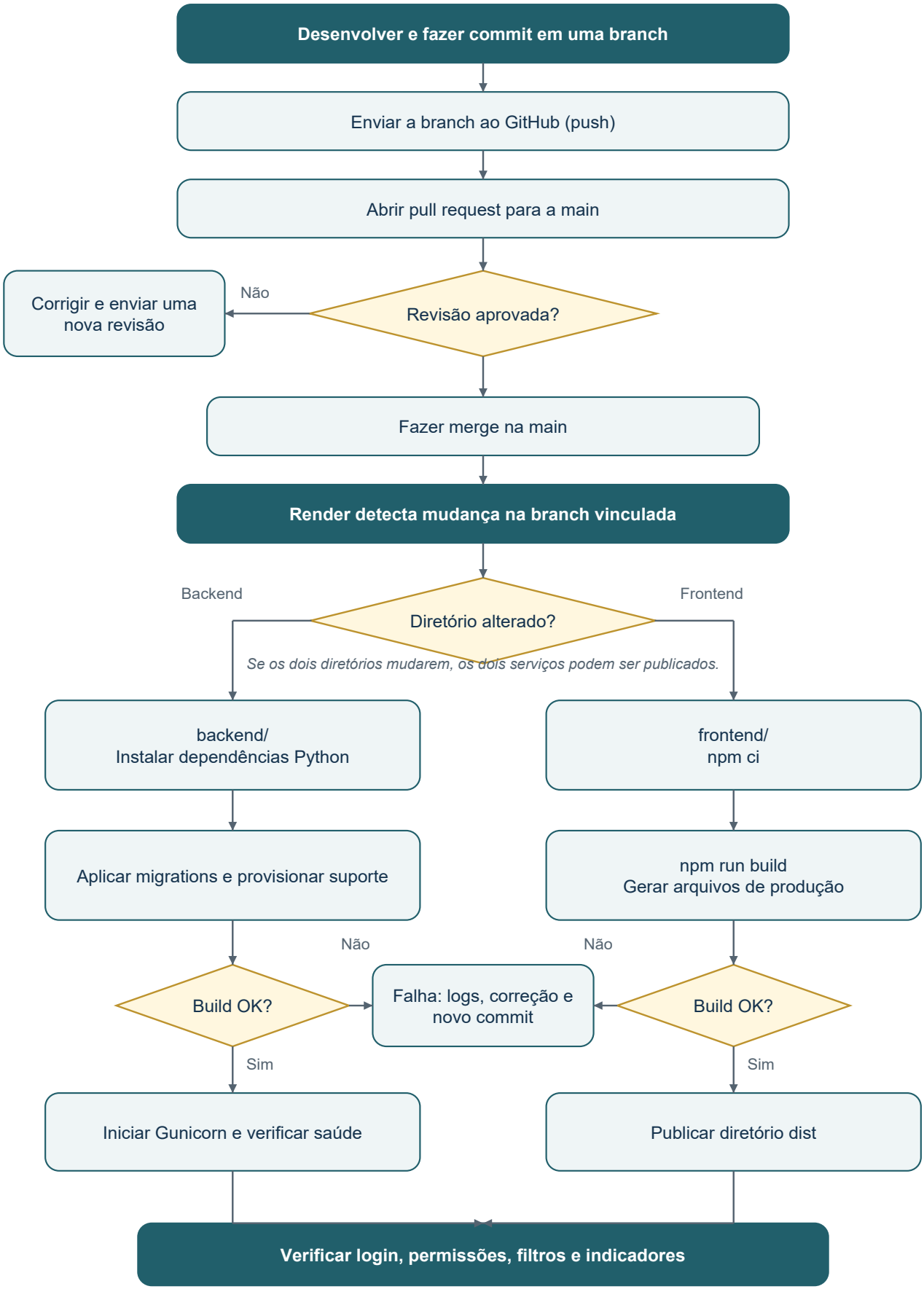
O Git registra as alterações localmente. O GitHub hospeda o repositório compartilhado. A Render obtém a revisão selecionada, executa os comandos de build configurados e publica a aplicação.

Configuração atual

Configuração	Backend	Frontend
Repositório	`DatumLex/datumlex-core`	`DatumLex/datumlex-core`
Branch de deploy	`main`	`main`
Serviço Render	`datumlex-demo-api`	`datumlex-demo-web`
Tipo	Serviço web Python	Site estático
Diretório raiz	`backend/`	`frontend/`
Deploy automático	**On Commit**	**On Commit**
Execução	Django e Gunicorn	Arquivos gerados por React/Vite

O vínculo com uma branch e o gatilho **On Commit** são descritos na documentação de deploy da Render.

Fluxograma


Desenvolver e fazer commit em uma branch
Enviar a branch ao GitHub (push)
Abrir pull request para a main
Revisão aprovada?
Não
Corrigir e enviar uma nova revisão
Fazer merge na main
Render detecta mudança na branch vinculada
Diretório alterado?
Backend
Frontend
Se os dois diretórios mudarem, os dois serviços podem ser publicados.
backend/
Instalar dependências Python
Aplicar migrations e provisionar suporte
Build OK?
Não
Sim
Iniciar Gunicorn e verificar saúde
frontend/
npm ci
npm run build
Gerar arquivos de produção
Build OK?
Não
Sim
Publicar diretório dist
Falha: logs, correção e novo commit
Verificar login, permissões, filtros e indicadores

O pull request e a revisão representam o fluxo de desenvolvimento pretendido pela equipe. O gatilho **On Commit** responde às mudanças que chegam à `main`, inclusive por push direto. A exigência de pull request revisado precisa ser aplicada separadamente nas regras do repositório GitHub.

Explicação do processo

1. Desenvolver e registrar

O desenvolvedor altera o backend, o frontend ou ambos. O commit registra essa mudança no Git, e o push a disponibiliza no GitHub.

Enviar apenas para uma branch de desenvolvimento não aciona os deploys atuais da `main`. A mudança precisa chegar à branch vinculada aos serviços Render.

2. Revisar e integrar

A equipe abre um pull request, revisa as alterações e integra o trabalho aprovado à `main`. A nova revisão fica disponível para os serviços de deploy.

A revisão de código e os testes automáticos fornecem evidências para a liberação. A configuração atual da Render, por si só, não exige um workflow GitHub Actions aprovado.

3. Identificar o serviço afetado

Os dois serviços utilizam o mesmo repositório, com diretórios raiz separados:

- Alterações em `backend/` podem acionar a publicação do backend.
- Alterações em `frontend/` podem acionar a publicação do frontend.
- Mudanças nos dois diretórios podem acionar os dois serviços.

Arquivos fora do diretório raiz de um serviço não acionam automaticamente seu deploy. Arquivos compartilhados fora desses diretórios exigem tratamento explícito. Consulte o guia de monorepos.

4. Construir e publicar o backend

O comando de build é:

```
pip install -r requirements.txt && \
python manage.py migrate --noinput && \
python manage.py bootstrap_support
```

Ele instala as dependências declaradas, aplica as migrations pendentes e garante a existência da conta de suporte. O provisionamento não redefine uma senha de suporte já existente.

Depois do build, a Render inicia o serviço com:

```
gunicorn config.wsgi:application --bind 0.0.0.0:$PORT \
  --workers 1 --timeout 120
```

O endereço de verificação de saúde é:

```
/api/health/
```

Os comandos executam em `backend/`. Conexões e segredos são fornecidos pelas variáveis de ambiente da Render, sem credenciais no código versionado.

As migrations executam durante o build porque o serviço gratuito não oferece comando de pré-deploy. Uma falha de publicação não significa que uma migration já aplicada foi desfeita. Alterações no banco exigem migrations compatíveis e um plano apropriado de backup e recuperação.

5. Construir e publicar o frontend

O comando de build é:

```
npm ci && npm run build
```

Ele executa em `frontend/`. A Render publica o diretório `dist`. A variável de build `VITE_API_BASE_URL` aponta o frontend para a API Django.

Como o Vite incorpora essa variável aos arquivos gerados, mudar a URL da API exige reconstruir o frontend.

6. Verificar a liberação

A equipe confere:

- Situação do deploy e logs na Render.
- Resposta do endpoint de saúde do backend.
- Login e permissões na aplicação.
- Filtros e indicadores afetados pela alteração.

Frontend e backend são publicados independentemente. Uma mudança nos dois serviços precisa manter compatibilidade durante o intervalo em que um deploy termina antes do outro.

Relação com a integração contínua

Mecanismo	Responsabilidade	Situação atual
Git e GitHub	Versionar e compartilhar o código.	Em uso
Revisão de pull request	Avaliar a mudança antes do merge.	Fluxo da equipe; exigência depende das regras do repositório
CI no GitHub Actions	Verificar automaticamente código, testes e build.	Ainda não é uma condição obrigatória deste deploy
Render On Commit	Construir e publicar quando a branch vinculada muda.	Ativo nos dois serviços
Render After CI Checks Pass	Aguardar a aprovação dos checks antes de publicar.	Melhoria futura, atualmente desativada

A melhoria pretendida é estabelecer verificações confiáveis e mudar a Render para **After CI Checks Pass**. Os checks suportados e o comportamento do gatilho estão descritos no guia de deploy.

Código e dados judiciais são processos separados

```
Código:
Commit Git -> GitHub main -> build Render -> deploy da aplicação

Dados judiciais:
DataJud / Pangea / STJ -> ETL -> PostgreSQL Render -> consultas da aplicação
```

O deploy automático atualiza o código. Ele não copia o banco Docker local, não executa automaticamente a coleta das fontes e não publica os textos judiciais locais.

O ETL possui execução e checkpoints próprios. Sua tarefa diária permanece desativada após a conclusão da carga inicial de 10.000 registros.

Por que essa automação se adapta ao DatumLex

A integração aproxima o deploy do fluxo GitHub da equipe, reduz etapas manuais repetitivas e oferece logs e histórico para diagnóstico. Os serviços separados refletem a estrutura do repositório, enquanto variáveis de ambiente mantêm a configuração fora do código. A exigência de CI testado e a validação controlada das liberações são os próximos passos para uma entrega mais confiável.